



TENTACLES: a consensus Machine Learning tool for robust biomarker discovery in heterogeneous data

Giorgio Montesi, Gabriel Dos Santos Mouta, Maria Novedrati, André Ferreira Cunha, Alessandro Fuschi, Simone Lucchesi, Chiara Sonnati, Annalisa Ciabattini, Francesco Santoro, Donata Medagliani, Helder Nakaya

Contents

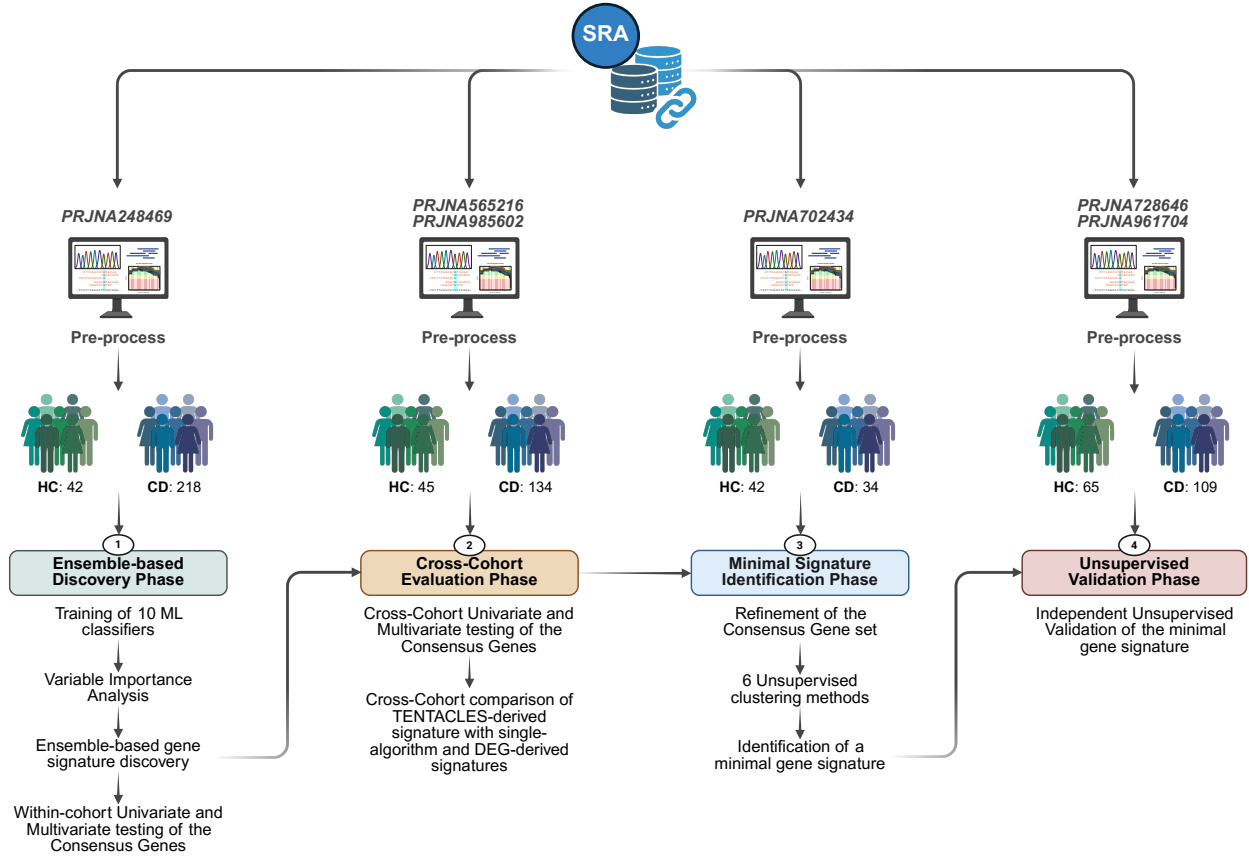
1	Introduction	3
1.1	Structure of the document	3
2	Dataset composition and preprocessing (<code>preProcess</code>)	5
2.1	Dataset 1 - <code>preProcess()</code>	6
2.2	Dataset 2 - <code>preProcess()</code>	7
2.3	Dataset 3 - <code>preProcess()</code>	7
2.4	Dataset 4 - <code>preProcess()</code>	7
3	Ensemble-based Signature Discovery Phase on Dataset 1 (<code>runClassifiers</code> and <code>getConsensus</code>)	8
3.1	Training multiple classification algorithms (<code>runClassifiers</code>)	8
3.2	Identification of consensus gene-signature through ensemble learning (<code>getConsensus</code>)	11
3.3	Differential gene expression (DEG) analysis on the Discovery Cohort	15
4	Cross-cohort Signature Evaluation on Dataset 2 (<code>testConsensus</code>)	17
4.1	Benchmarking against Single-Algorithm and Differential Expression-based methods	18
4.2	Reproducibility of differentially expressed genes across cohorts	20
5	Minimal Signature Identification Phase on Dataset 3 (<code>valConsensus</code>)	22
5.1	Signature refinement based on cross-cohort importance concordance	22
5.2	Use of <code>valConsensus()</code> to identify minimal gene signature	23

6	Independent Unsupervised Validation on Dataset 4	25
7	Session Info	27

1 Introduction

This vignette demonstrates the application of **TENTACLES** to bulk RNA-seq datasets from patients with Crohn's disease (CD) and healthy controls (HC), reproducing the analysis described in the manuscript "*TENTACLES: a consensus Machine Learning tool for robust biomarker discovery in heterogeneous data.*"

The analysis is organized into four sequential phases, each applied to an independent cohort, as illustrated below:



1.1 Structure of the document

- **Dataset composition and preprocessing (preProcess)**

Preparation of libraries and datasets required to run the TENTACLES workflow

- Preprocessing of Dataset 1 with `preProcess()` (normalization and filtering)
- Preprocessing of Dataset 2 with `preProcess()` (normalization, filtering and batch correction)
- Preprocessing of Dataset 3 with `preProcess()` (normalization and filtering)
- Preprocessing of Dataset 4 with `preProcess()` (normalization, filtering and batch correction)

- **Ensemble-based Signature Discovery Phase on Dataset 1 (runClassifiers and getConsensus)**

- Training multiple classification algorithms (`runClassifiers`)
- Identification of consensus gene-signature through ensemble learning (`getConsensus`)

- DEG Analysis on the Discovery cohort
- **Cross-cohort Signature Evaluation Phase on Dataset 2 (`testConsensus`)**
 - Benchmarking against Single-Algorithm and Differential Expression-based methods
 - Reproducibility of differentially expressed genes across cohorts
- **Minimal Signature Identification Phase on Dataset 3 (`valConsensus`)**
 - Signature refinement based on cross-cohort importance concordance
 - Use of `valConsensus()` to identify minimal gene signature
- **Independent Unsupervised Validation on Dataset 4**
 - k-means clustering to assess Minimal Signature unsupervised classification capacity

2 Dataset composition and preprocessing (preProcess)

This section imports the required libraries and the datasets used throughout the analysis.

```
# Increase max size for parallel processing
options(future.globals.maxSize = 2 * 1024^3)

# Load the package
library(tiff)
library(grid)
library(Rtsne)
library(cowplot)
library(scales)
library(tibble)
library(ggrepel)
library(DESeq2)
library(tidyverse)
library(TENTACLES)
```

The datasets used in this vignette correspond to publicly available intestinal bulk RNA-Seq samples from Crohn's disease (CD) patients and healthy controls (HC). Raw sequencing data were originally retrieved from the NCBI Sequence Read Archive (SRA) under the following BioProjects:

- *PRJNA248469* – **Dataset 1** (Ensemble-based Signature Discovery Cohort)
- *PRJNA565216* and *PRJNA985602* – **Dataset 2** (Cross-cohort Signature Evaluation Cohort)
- *PRJNA702434* – **Dataset 3** (Minimal Signature Identification Cohort)
- *PRJNA728646* and *PRJNA961704* – **Dataset 4** (Independent Unsupervised Validation Cohort)

All samples were selected based on the inclusion criteria described in the manuscript: intestinal biopsies from individuals clinically diagnosed with CD or from HC.

In the original study, the raw FASTQ files were processed through a uniform bioinformatics pipeline to ensure cross-study consistency. Briefly, quality control was performed with **fastp** (v1.0.1), reads were aligned to the GRCh38.p14 human reference genome using **STAR** (v2.7.10), strandedness was verified with **RSeQC** (v5.0.1), and gene-level quantification was obtained with **Rsubread** (v2.22.1).

For the present vignette, these processed count matrices and their sample metadata were re-used as a single ready-to-load object (**ibd_dataset.RData**, containing **count_d1** through **count_d4** (samples on rows, genes on columns) and **desc_d1** through **desc_d4** (sample-level annotations)), deposited on Zenodo as cited in the paper.

```
# Load all datasets
load("ibd_dataset.RData")
```

The **preProcess()** function from **TENTACLES** performs library-size normalization, low-expression filtering, and (if specified) batch-effect correction.

It requires as input:

- **df.count**: a data frame of raw gene counts, with samples as rows and genes as columns;
- **df.clin**: a data frame containing sample-level metadata, with samples as rows and variables (e.g., class labels, batch ID) as columns;

- **class**: the name of the column in `df.clin` indicating the outcome variable (e.g., "class"), which must be a binary factor (e.g., "CD" vs "Not IBD").

Optional arguments allow for batch correction (via `batch` and `covar.mod`), plotting (PCA and PVCA before and after batch correction), and threshold customization for low gene expression filtering (through `mincpm` and `minfraction`).

The `is.normalized` parameter specifies whether the input data are already normalized: it should be set to `TRUE` for microarray data and `FALSE` (default) for RNA-Seq data.

In this case, since the input consists of raw RNA-Seq counts, we leave `is.normalized = FALSE`.

The output of the `preProcess()` function is a `preProcess.obj` object, a structured S4 object containing all intermediate and final elements from the preprocessing pipeline.

Its main slots include:

- **raw**: the original count matrix provided as input;
- **processed**: a list of data frames containing the results of each preprocessing step, including:
- **normalized**: the library-size normalized counts data frame;
- **filtered.data**: genes passing expression thresholds;
- **adjusted.data**: the final matrix used for classification (after optional batch correction);
- **metadata**: a data frame with sample-level annotations, filtered and aligned to the final count matrix;
- **data.info**: a list with metadata about the dataset, including data type ("`RNA-Seq`" or "`microarray`"), normalization status, number of samples and features, and whether batch correction was applied.

This object is compatible with all downstream TENTACLES functions (e.g., `runClassifiers()`, `testConsensus()`, `valConsensus()`), ensuring reproducibility and consistent sample tracking.

The resulting `preProcess.obj` provides the input structure for the multi-model training step described next.

2.1 Dataset 1 - `preProcess()`

```
# Preprocess dataset 1 with preProcess()
pp_d1 <- preProcess(df.count = count_d1, df.clin = desc_d1, class = "class",
                    case.label = "CD", is.normalized = FALSE,
                    mincpm = 1, minfraction = 0.1, plot = FALSE)

str(pp_d1, list.len = 4)
#> Formal class 'preProcess.obj' [package "TENTACLES"] with 4 slots
#> ..@ raw      : 'data.frame': 260 obs. of 10062 variables:
#> .. ..$ TNFRSF4      : int [1:260] 120 53 146 144 56 74 34 178 101 30 ...
#> .. ..$ TNFRSF18     : int [1:260] 36 53 111 106 30 93 49 107 57 38 ...
#> .. ..$ PRDM16       : int [1:260] 45 34 7 46 9 27 9 13 18 29 ...
#> .. ..$ PRKCZ        : int [1:260] 432 551 232 390 176 263 142 178 154 642 ...
#> .. .. [list output truncated]
#> ..@ processed: List of 2
#> .. ..$ normalized  : 'data.frame': 260 obs. of 8426 variables:
```

```

#> .. .. .$. TNFRSF4      : num [1:260] 4.2 3.24 4.91 4.03 3.9 ...
#> .. .. .$. TNFRSF18     : num [1:260] 2.63 3.24 4.53 3.62 3.09 ...
#> .. .. .$. PRDM16       : num [1:260] 2.91 2.68 1.26 2.56 1.7 ...
#> .. .. .$. PRKCZ        : num [1:260] 5.99 6.47 5.56 5.41 5.49 ...
#> .. .. . [list output truncated]
#> .. .. $.adjusted.data:'data.frame': 260 obs. of  8426 variables:
#> .. .. .$. TNFRSF4      : num [1:260] 4.2 3.24 4.91 4.03 3.9 ...
#> .. .. .$. TNFRSF18     : num [1:260] 2.63 3.24 4.53 3.62 3.09 ...
#> .. .. .$. PRDM16       : num [1:260] 2.91 2.68 1.26 2.56 1.7 ...
#> .. .. .$. PRKCZ        : num [1:260] 5.99 6.47 5.56 5.41 5.49 ...
#> .. .. . [list output truncated]
#> ..@ metadata :'data.frame': 260 obs. of  12 variables:
#> .. .. $. Age           : num [1:260] 12.5 8 9 5.08 10.42 ...
#> .. .. $. Assay.Type     : chr [1:260] "RNA-Seq" "RNA-Seq" "RNA-Seq" "RNA-Seq" ...
#> .. .. $. Study          : chr [1:260] "PRJNA248469" "PRJNA248469" "PRJNA248469" "PRJNA248469" ...
#> .. .. $. ID             : chr [1:260] "SAMN03322967" "SAMN03322968" "SAMN03322969" "SAMN03322970" ...
#> .. .. [list output truncated]
#> ..@ data.info:List of 2
#> .. .. $. type           : chr "rnaseq"
#> .. .. $. normalized: logi FALSE

```

2.2 Dataset 2 - preProcess()

Since the two studies in Dataset 2 originate from different experiments, they introduce batch effects that must be addressed before downstream analysis. The variable `batch_new` was constructed by combining the study identifier with the internal batch structure of each study, such that samples from the same study but different internal batches receive distinct batch labels, and was passed to the `batch` argument of `preProcess()`.

```

# Preprocess dataset 2 with preProcess()
pp_d2 <- preProcess(df.count = count_d2, df.clin = desc_d2, batch = "batch_new",
                    class = "class", covar.mod = "class", case.label = "CD", plot = FALSE)

```

2.3 Dataset 3 - preProcess()

```

# Preprocess dataset 3 with preProcess()
pp_d3 <- preProcess(df.count = count_d3, df.clin = desc_d3,
                    class = "class", case.label = "CD")

```

2.4 Dataset 4 - preProcess()

Since Dataset 4 was assembled by merging two independent studies, batch effects were addressed before downstream analysis. The study identifier was passed to the `batch` argument of `preProcess()`, with the biological class variable included as a covariate to preserve true signal.

```

# Preprocess dataset 4 with preProcess()
pp_d4 <- preProcess(df.count = count_d4, df.clin = desc_d4, batch = "Study",
                    class = "class", covar.mod = "class", plot = FALSE, case.label = "CD")

```

3 Ensemble-based Signature Discovery Phase on Dataset 1 (`runClassifiers` and `getConsensus`)

In this section, we focus on the preprocessed Dataset 1 (*PRJNA248469*), which serves as the Ensemble-based Signature Discovery Cohort for the construction of a robust gene signature distinguishing CD from HC.

The analysis proceeds through the following steps:

1. **Classifier training and feature selection**

A panel of supervised machine-learning models is trained using the `runClassifiers()` function. Each model is coupled with a dedicated feature selection strategy, executed within resampling folds to avoid information leakage. Variable importance scores are computed for all classifiers.

2. **Consensus signature extraction**

Features that are recurrently selected across multiple high-performing models are aggregated into a unified consensus gene-signature using the `getConsensus()` function.

3.1 Training multiple classification algorithms (`runClassifiers`)

The function `runClassifiers()` trains, tunes, and evaluates several supervised models through a unified interface. Each classifier can be coupled with a specific feature-selection strategy, and model hyperparameters are optimized through cross-validation. Variable importance (VIP) is computed for each model, with a permutation-based fallback when direct computation is unavailable.

It takes as inputs:

- **preProcess.obj**: output of `preProcess()` containing the adjusted counts and aligned metadata. Alternatively, `df.count` and `df.clin` can be passed directly if preprocessing was performed outside TENTACLES.
- **models**: character vector of classifiers to fit. Supported options include: "xgboost", "bag_tree", "lightGBM", "pls", "logistic", "C5_rules", "mars", "bag_mars", "mlp", "bag_mlp", "decision_tree", "rand_forest", "svm_linear", "svm_poly", "svm_rbf".
- **selector.recipes**: feature-selection strategy for each model. Supported: "base", "boruta", "roc", "infgain", "mrmr", "corr". If a single recipe is provided, it is applied to all models; otherwise, pairing follows position order.
- **tuning.method**: hyperparameter search. Options: "tune_grid" (default), "tune_race_anova", "tune_race_win_loss", "tune_bayes", "tune_sim_anneal".
- **n**: an integer specifying the number of iterations for the tuning method.
- **v**: an integer specifying the number of folds for the cross-validation during the hyperparameters tuning.
- **Thresholds and selectors**: `selector.threshold` (threshold parameter used for "roc", "infgain", "mrmr", "corr"), `boruta.maxRuns` (for "boruta").
- **Optional Balancing and filtering**: `downsample = TRUE` to downsample the majority class; `filter = TRUE` to remove genes not annotated in GO/KEGG databases before modeling.
- **Other controls**: `metric` (default "accuracy"), `nsim` (# simulations for permutation VIP), `seed`, `plot`, and `parallel` (to speed-up computation).

In the manuscript, ten models were evaluated with model-specific selectors and 5-fold CV grid search. The optimal configuration per model was chosen by the highest F1-score across folds. The `filter = TRUE` argument restricts the analysis to genes with annotations in GO or KEGG databases, limiting the feature space to approximately 10,000 biologically interpretable genes and reducing computational load. This filter is optional and can be disabled by setting `filter = FALSE` to retain all expressed genes.

```
# runClassifiers()
rc <- runClassifiers(
  pp_d1,
  models = c("C5_rules", "pls", "rand_forest", "bag_mlp", "decision_tree", "mlp",
             "svm_linear", "xgboost", "lightGBM", "mars"),
  selector.recipes = c("boruta", "boruta", "corr", "roc", "boruta", "roc",
                      "boruta", "base", "roc", "boruta"),
  tuning.method = "tune_grid",
  filter = TRUE, downsample = TRUE,
  selector.threshold = 0.95, boruta.maxRuns = 100,
  parallel = TRUE,
  metric = "f_meas",
  n = 5, v = 5, seed = 1789,
  plot = FALSE
)
```

The function returns a S4 object of class `runClassifiers.obj` with:

- **data**: adjusted data and metadata used for modeling.
- **models.info**: finalized workflows per model (recipes + model + tuned parameters).
- **model.features**: variable importance tables for each model.
- **performances**: tuning curves and final metrics for each model.
- **predictions**: out-of-fold predictions and associated probabilities, when available.

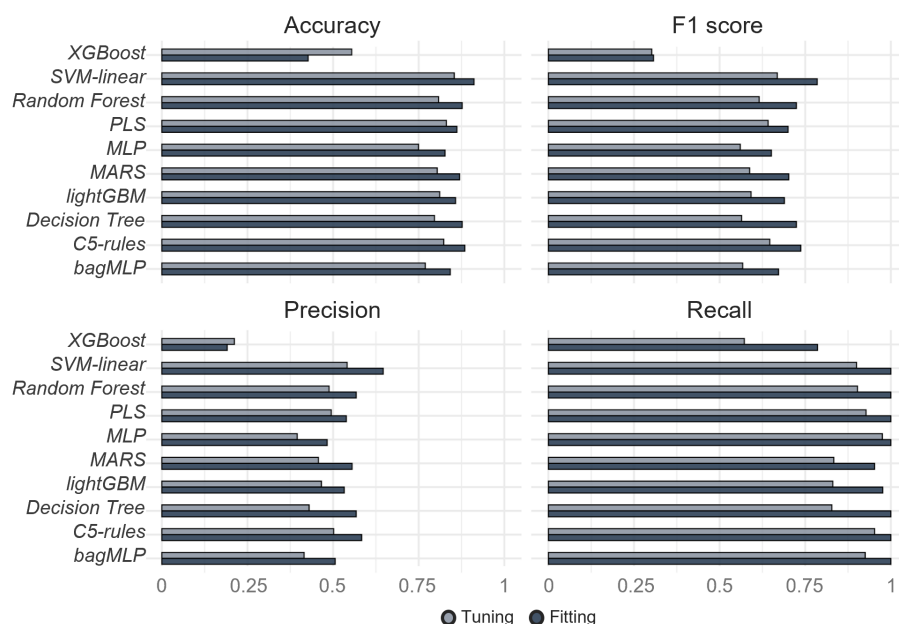
3.1.1 Visualizing model performance and feature overlap

After training, TENTACLES provides dedicated visualization utilities to interpret model performance and feature-selection overlap. Three main plotting utilities are available for objects returned by `runClassifiers()`.

The `performances.plot()` function visualizes the cross-validation and final metrics for each trained model. It takes as input the `runClassifiers.obj` object and returns a grouped bar plot summarizing **Accuracy**, **F1-score**, **Precision**, and **Recall**.

Light-colored bars represent performance during cross-validation, whereas dark bars correspond to the final model fitted on the entire training set:

```
performances.plot(rc)
```



To examine how individual samples are classified by each model, the `wrong.preds.plot()` function can be used. It takes the `@predictions` slot of the `runClassifiers.obj` object and produces a concordance plot where each column represents a sample and each row a classifier.

Red dots indicate misclassified samples, and grey dots denote correct predictions. This visualization highlights samples that are consistently difficult to classify across multiple algorithms:

```
wrong.preds.plot(rc@predictions) +
  theme( axis.text.x = element_text( angle = 45, hjust = 1, vjust = 1,
    margin = margin(t = 2),
    size = 6,
    color = "#7c7b7b"),
    plot.margin = margin(t = 10, r = 10, b = 5, l = 10)
  )
```



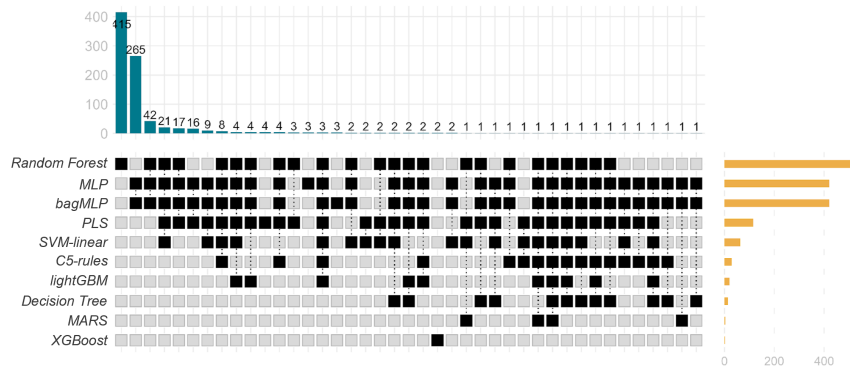
Note: Because these plots are built on `ggplot2`, they can be fully customized with standard `theme()` or aesthetic arguments, as shown above (e.g., adjusting text size or margins to better fit in documents).

The `upset.plot()` function visualizes the intersection of important genes across models. It takes the `runClassifiers.obj` object and generates an UpSet plot showing how feature sets selected by different classifiers intersect.

Each horizontal row represents a model, and each vertical bar shows the size of an intersection (number of genes selected by that specific combination of models).

This provides a concise overview of shared vs. model-specific gene selections, helping identify robust features contributing to the consensus signature:

`upset.plot(rc)`



3.2 Identification of consensus gene-signature through ensemble learning (`getConsensus`)

After training individual models, it is possible to derive a consensus set of features repeatedly appearing across high performing classifiers derived and computed in the training part through the `runClassifiers()` function.

To do this we use the `getConsensus()` function which takes as input a `runClassifiers.obj` or a binary data frame and supports 3 possible approaches:

- **Threshold-based selection:** assign values to `n_min` and `exclude` parameters. In this way we can exclude algorithms with poor performances and pick as consensus genes, those that appeared in at least a number of `n_min` algorithms.
- **Group-wise logic (single group):** leave as NULL these 2 parameters and specify instead `group1` and `meth1` parameters. In this way we can take the 'union' or the 'intersection' of the algorithms listed in `group1`.

- **Group-wise logic (two groups):** leave as NULL `n_min` and `exclude` while specifying `group1`, `group2`, `meth1`, `meth2` and `meth_comb` parameters all together. In this way we can take genes based on the `meth1` of `group1`, `meth_comb` with the `meth2` of `group2`. e.g. take the union (`meth1 = "union"`) of genes in `group1`, the union (`meth2 = "union"`) of genes in `group2` and pick the intersection (`meth_comb = "intersect"`) of these two groups.

In this analysis, we used the threshold-based approach and retrieved genes that were appearing in at least six algorithms (`n.min = 6`) excluding XGBoost (`exclude = c('xgboost_base')`):

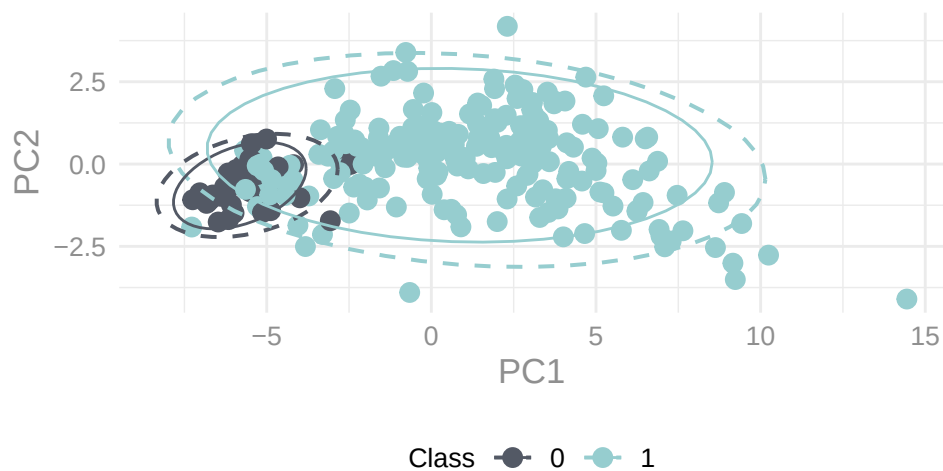
```
# getConsensus()
gc <- getConsensus(
  rc,
  n.min = 6,
  exclude = c("xgboost_base"),
  plot = FALSE
)
```

To analyze and visualize the discriminative performance of the genes selected by `getConsensus()`, the output object provides access to three different plots, each representing a complementary analysis of the consensus gene set.

These visualizations assess both multivariate and univariate behavior of the selected genes on the dataset used as input to `getConsensus()`.

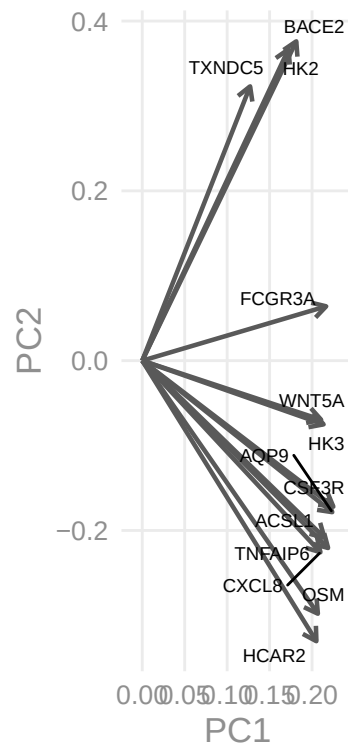
- `getConsensus.obj$test_consensus@pca$plots$pca_plot` displays a **Principal Component Analysis (PCA)** of the samples based on the consensus genes, showing overall class separation.

```
gc$test_consensus@pca$plots$pca_plot
```



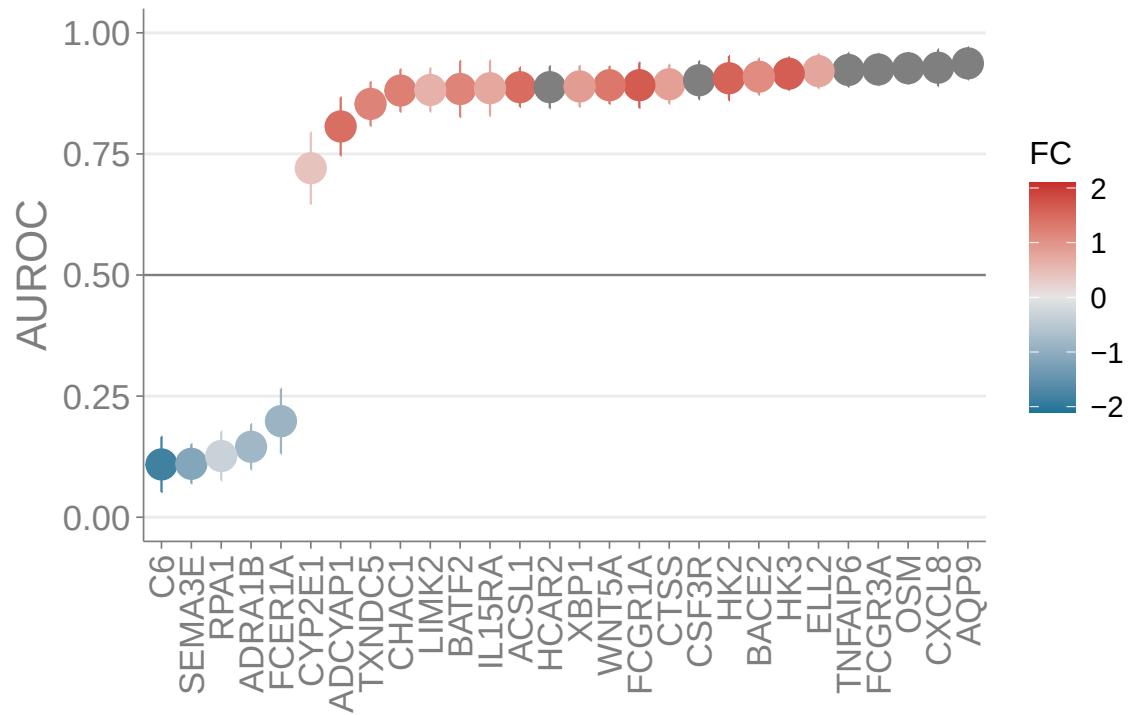
- `getConsensus.obj$test_consensus@pca$plots$loadings_plot` shows the **PCA loadings**, highlighting the contribution of each gene to the principal components and its recurrence across models.

```
gc$test_consensus@pca$plots$loadings_plot
```



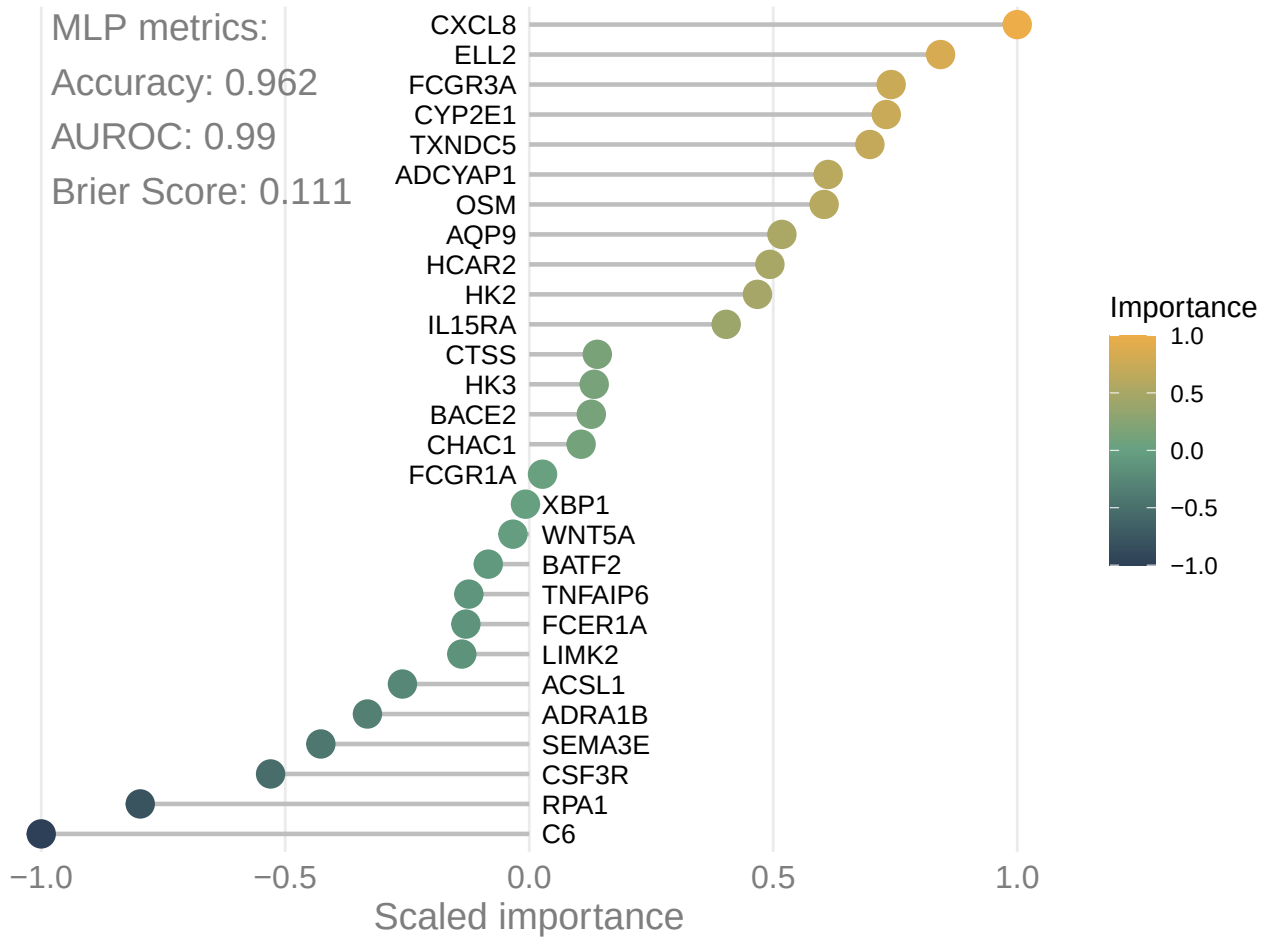
- `getConsensus.obj$test_consensus@auroc_fc$plot` summarizes **univariate performance** by plotting the AUROC of each gene, with colors indicating log2 fold change between classes.

```
gc$test_consensus@auroc_fc$plot
```



- `getConsensus.obj$test_consensus@mlp$plot` reports the performance of a **multivariate MLP classifier** trained only on the consensus genes, including classification metrics and variable importance.

```
gc$test_consensus@mlp$plot
```



3.3 Differential gene expression (DEG) analysis on the Discovery Cohort

To complement the workflow, we conducted a classical differential expression (DEG) analysis on the training cohort (Dataset 1 – *PRJNA248469*). The goal was to identify genes significantly modulated between CD and HC, to later compare their discriminative performance against the **TENTACLES** consensus-derived signature.

Raw counts were filtered for low-coverage samples (total reads < 500,000), normalized with DESeq2, and modeled with the formula `~ class`. Genes were considered differentially expressed if `padj < 0.05` and `|log2FC| >= 1`. The resulting DEG list (hereafter `genes_dds_d1`) served as a baseline reference for benchmarking:

```
# Transpose the gene count matrix, align sample names with clinical data and filter
x <- t(count_d1)
desc <- desc_d1
m <- match(desc$ID, colnames(x))
x <- x[, m]

# Remove samples with low total counts (<500,000) and update clinical data
keepsamples <- colSums(x) > 500000
x <- x[, keepsamples]
desc <- desc[keepsamples, ]

# Clean and prepare class and batch columns in clinical data
desc$class <- as.factor(desc$class)
```

```

desc$class <- relevel(desc$class, ref = "Not IBD") # Set "Not IBD" as reference
desc$batch2 <- as.factor(paste(desc$batch, desc$internal.batch)) # Combine batch info
desc$Group <- as.factor(paste(desc$batch, desc$class)) # Create group column

# ----- DESeq2 Analysis -----
# Create DESeq2 dataset object using count data and clinical metadata and subset
dds_d1 <- DESeqDataSetFromMatrix(countData = x, colData = desc, design = ~ class)

keep <- rownames(dds_d1) %in% colnames(pp_d1@processed$adjusted.data)
dds_d1 <- dds_d1[keep, ]

# Perform DESeq2 normalization and differential expression analysis
dds_d1 <- DESeq(dds_d1)

# Retrieve and subset DEG results
res_dds_d1 <- results(dds_d1, alpha = 0.05)
res_dds_d1 <- subset(res_dds_d1, padj < 0.05 & abs(log2FoldChange) >= 1)

# Define the final DEGs list
genes_dds_d1 <- rownames(res_dds_d1)

# Clean up unnecessary objects from the environment
rm(x, m, keep, keepsamples, desc)

```

4 Cross-cohort Signature Evaluation on Dataset 2 (`testConsensus`)

To evaluate the cross-cohort generalizability of the consensus gene-signature identified in the Discovery Cohort (Dataset 1), we use the preprocessed Dataset 2.

We then apply `testConsensus()` to this combined dataset to evaluate the cross-cohort signature discriminatory capability both at the univariate and multivariate level.

This function performs a comprehensive analysis of the input genes on the dataset passed as input, combining univariate and multivariate evaluations.

Specifically, `testConsensus()` includes the following analysis:

- **Principal Component Analysis (PCA):**
Dimensionality reduction based on the consensus genes.
The top `top_loadings` genes (by absolute loading) are identified, and the PCA scores and loadings are returned showing class separation.
- **AUROC and Fold Change analysis:**
For each gene, the function computes the Area Under the ROC Curve (AUROC) and the log2 fold change between classes. These metrics are summarized in a data frame and visualized in a joint plot.
- **Heatmap with hierarchical clustering (HC):**
A normalized expression heatmap of the consensus genes, grouped by hierarchical clustering, to visualize co-expression and class-specific patterns.
- **Multi-Layer Perceptron (MLP):**
A multivariate model is trained using the consensus gene set on the input Dataset.
The resulting output includes the model's performance metrics and feature importance scores.

The output is an S4 object of class `testConsensus.obj`, containing all intermediate data structures and `ggplot2` objects for full customization and reuse.

To assess the discriminatory capacity in an independent cohort, we apply `testConsensus()` to the preprocessed Dataset 2, using the consensus genes identified from Dataset 1 as input.

```
# External validation via testConsensus()
tc <- testConsensus(df.count = pp_d2@processed$adjusted.data,
                    gene.list = gc$consensusGenes,
                    class = pp_d2@metadata$class,
                    plot = FALSE)
```

The output of `testConsensus()` is a structured S4 object of class `testConsensus.obj`, which includes:

- **data:** a list with the input expression matrix (`df.count`), gene list (`gene.list`), and class labels (`class`).
- **pca:** PCA results, including sample scores, gene loadings, variance explained, and the corresponding PCA plot.
- **auroc_fc:** per-gene AUROC** and **log2 fold change values, alongside a summary plot.
- **heatmap:** a heatmap with hierarchical clustering of the consensus genes.
- **mlp:** a multivariate MLP classifier, with model performance metrics, variable importances, and an associated plot.

The resulting visualizations can be accessed directly from the object using:

```
tc@pca$plots$pca_plot
tc@pca$plots$loadings_plot
tc@auroc_fc$plot
tc@mlp$plot
```

4.1 Benchmarking against Single-Algorithm and Differential Expression-based methods

To contextualise the discriminative performance of the TENTACLES consensus signature relative to conventional biomarker discovery approaches, a benchmarking analysis was performed on Dataset 2 using the same MLP-based `testConsensus()` framework.

Two classes of comparators were evaluated: DESeq2-derived gene sets of increasing size assembled from Dataset 1 by ranking genes by ascending adjusted p-value (top 28, 50, 100, 200, 300, 400, and all 582 DEGs), and the full set of genes identified as important by each individual classifier trained during the Discovery Phase.

Each comparator was evaluated on Dataset 2 using `testConsensus()`, recording Accuracy, ROC-AUC, and Brier score under identical conditions.

```
# ML model signatures
res_perf <- imap_dfr(rc@model.features, ~
  testConsensus(df.count = pp_d2@processed$adjusted.data,
    gene.list = .x$Variable,
    class = pp_d2@metadata$class,
    plot = FALSE)@mlp$test_performance %>%
    mutate(modelo = .y)
) %>%
  full_join(tc@mlp$test_performance %>% mutate(modelo = "original_d2"))

# DESeq2 signatures
genes_rank <- as.data.frame(results(dds_d1)) %>%
  rownames_to_column("gene") %>%
  arrange(padj)

res_perf_deseq <- map_dfr(c(28, 50, 100, 200, 300, 400, 582), ~
  testConsensus(df.count = pp_d2@processed$adjusted.data,
    gene.list = slice_head(genes_rank, n = .x) %>% pull(gene),
    class = pp_d2@metadata$class,
    plot = FALSE)@mlp$test_performance %>%
    mutate(modelo = paste0("DESeq_top_", .x))
)

# Combine
res_performance <- full_join(res_perf, res_perf_deseq)
rm(res_perf, res_perf_deseq)

#Table
labels_models <- c(
  decision_tree_boruta = "Decision Tree", original_d2 = "TENTACLES",
  bag_mlp_roc = "Bagged MLP", mlp_roc = "MLP",
  pls_boruta = "PLS", rand_forest_corr = "Random Forest",
```

```

svm_linear_boruta = "Linear SVM", C5_rules_boruta = "C5 Rules",
mars_boruta      = "MARS", lightGBM_roc        = "LightGBM",
xgboost_base     = "XGBoost", DESeq_top_28     = "DEG - Top 28",
DESeq_top_50     = "DEG - Top 50 ", DESeq_top_100 = "DEG - Top 100 ",
DESeq_top_200    = "DEG - Top 200 ", DESeq_top_300 = "DEG - Top 300 ",
DESeq_top_400    = "DEG - Top 400 ", DESeq_top_582 = "DEG all set")

# Build gene counts vector
n_genes <- c(
  map_int(rc@model.features, ~ length(.x$Variable)),
  original_d2 = length(gc$consensusGenes),
  setNames(c(28, 50, 100, 200, 300, 400, 582),
    paste0("DESeq_top_", c(28, 50, 100, 200, 300, 400, 582)))
)

res_performance %>%
  filter(.metric %in% c("accuracy", "roc_auc", "brier_class")) %>%
  select(modelo, .metric, .estimate) %>%
  pivot_wider(names_from = .metric, values_from = .estimate) %>%
  mutate(
    n_genes      = n_genes[modelo],
    modelo       = recode(modelo, !!!labels_models),
    brier_inv    = 1 - brier_class,
    aggregated_score = round(accuracy + roc_auc + brier_inv, 2),
    accuracy     = round(accuracy, 2),
    roc_auc      = round(roc_auc, 2),
    brier_label  = paste0(round(brier_class, 2), " (", round(brier_inv, 2), ")")
  ) %>%
  select(modelo, n_genes, accuracy, roc_auc, brier_label, aggregated_score) %>%
  dplyr::rename(
    "Model"           = "modelo",
    "`Number of important genes`" = "n_genes",
    "Accuracy"        = "accuracy",
    "`ROC-AUC`"       = "roc_auc",
    "`Brier Score (1 - Brier)`" = "brier_label",
    "`Aggregated Score`" = "aggregated_score"
  ) %>%
  knitr::kable(align = "lcccc")

```

Model	Number of important genes	Accuracy	ROC-AUC	Brier Score (1 - Brier)	Aggregated Score
C5 Rules	29	0.73	0.79	0.19 (0.81)	2.33
PLS	116	0.62	0.63	0.25 (0.75)	2.00
Random Forest	540	0.65	0.73	0.19 (0.81)	2.19
Bagged MLP	421	0.64	0.73	0.2 (0.8)	2.16
Decision Tree	14	0.75	0.87	0.19 (0.81)	2.43
MLP	421	0.64	0.73	0.2 (0.8)	2.16
Linear SVM	64	0.76	0.83	0.18 (0.82)	2.41

Model	Number of important genes	Accuracy	ROC-AUC	Brier Score (1 - Brier)	Aggregated Score
XGBoost	2	0.73	0.56	0.21 (0.79)	2.07
LightGBM	20	0.71	0.67	0.19 (0.81)	2.19
MARS	4	0.73	0.66	0.2 (0.8)	2.20
original_g25	NA	0.80	0.82	0.18 (0.82)	2.44
DEG - Top 28	28	0.75	0.82	0.19 (0.81)	2.38
DEG - Top 50	50	0.78	0.76	0.17 (0.83)	2.37
DEG - Top 100	100	0.76	0.77	0.18 (0.82)	2.35
DEG - Top 200	200	0.65	0.62	0.23 (0.77)	2.05
DEG - Top 300	300	0.64	0.70	0.21 (0.79)	2.13
DEG - Top 400	400	0.58	0.62	0.24 (0.76)	1.96
DEG all set	582	0.58	0.71	0.2 (0.8)	2.09

4.2 Reproducibility of differentially expressed genes across cohorts

Finally, we also performed an independent DEG analysis on the merged Dataset 2.

The following code reproduces this step, filtering low-count samples and including both `class` and `batch2` as factors in the model design:

```
x <- t(count_d2)
desc <- desc_d2
m <- match(desc$ID, colnames(x))

x <- x[, m]
desc$ID == colnames(x)

keepsamples <- colSums(x) > 500000
x <- x[, keepsamples]
desc <- desc[keepsamples, ]

desc$class <- as.factor(desc$class)
desc$class <- relevel(desc$class, ref = "Not IBD")
desc$batch2 <- as.factor(paste(desc$batch, desc$internal.batch))
desc$Group <- as.factor(paste(desc$batch, desc$class))

dds_d2 <- DESeqDataSetFromMatrix(
  countData = x,
  colData = desc,
  design = ~ class + batch2
)

keep <- rownames(dds_d2) %in% colnames(pp_d2@processed$adjusted.data)
dds_d2 <- dds_d2[keep, ]
dds_d2 <- DESeq(dds_d2)
```

```

res_dds_d2 <- results(dds_d2, alpha = 0.05)

res_dds_d2 <- subset(res_dds_d2,
                     padj < 0.05 &
                     abs(log2FoldChange) >= 1)

genes_dds_d2 <- rownames(res_dds_d2)

rm(x, m, keep, keepsamples, desc)

```

To assess the overlap among the TENTACLES consensus gene-signature, the DEG set derived from Dataset 1, and the DEG set derived from Dataset 2, the pairwise intersections were computed as follows:

```

# Genes shared between DEG sets from Dataset 1 and Dataset 2
intersection_deseqs <- intersect(genes_dds_d1, genes_dds_d2)

# Number of shared DEGs
length(intersection_deseqs)
#> [1] 385

# Overlap between shared DEGs and the TENTACLES consensus gene-signature
intersect(intersection_deseqs, gc$consensusGenes)
#> [1] "FCGR3A" "CSF3R" "HK2" "TNFAIP6" "WNT5A" "CXCL8" "ACSL1"
#> [8] "HK3" "ADRA1B" "C6" "TXNDC5" "SEMA3E" "BATF2" "HCAR2"
#> [15] "AQP9" "OSM" "BACE2"

```

5 Minimal Signature Identification Phase on Dataset 3 (`valConsensus`)

The `valConsensus()` function provides an unsupervised validation framework for assessing the discriminative power of a gene signature on a new dataset, without fitting supervised models.

Given an expression matrix (`df.count`) and a set of input genes (`gene.list`), the function evaluates the clustering performance of multiple combinations of genes across a panel of unsupervised algorithms.

Specifically, the following clustering methods are implemented:

- **K-Means**
- **Gaussian Mixture Models (GMM)**
- **Hierarchical Clustering**
- **K-Means on PCA-reduced dimensions**
- **K-Means on t-SNE-reduced dimensions**
- **K-Means on UMAP-reduced dimensions**

For each method and gene combination, the function computes standard classification metrics by comparing cluster assignments to the known class labels (`labels` vector parameter), including Accuracy, Precision, Recall and F1 Score (default metric for ranking).

The user can also specify:

- `N`: the number of top-performing gene combinations to retain (default = 10),
- `metric`: the evaluation criterion to prioritize combinations ("Accuracy", "Precision", "Recall", or "FScore").

5.1 Signature refinement based on cross-cohort importance concordance

The within-cohort MLP models fitted on Dataset 1 and Dataset 2 compute variable importance via the Olden algorithm, which yields a signed scalar for each gene: positive values indicate a contribution toward CD prediction, negative values toward HC. A gene was retained in the refined set if and only if its Olden importance score carried the same sign in both cohorts, indicating a consistent and reproducible directional contribution.

Of the 28 consensus genes, 14 met this criterion and were carried forward to the Minimal Signature Identification Phase. The remaining 14 showed discordant signs, likely reflecting dataset-specific covariates rather than genuinely opposing biological roles. `valConsensus()` was applied to this refined subset:

```
#Analysis----
# Joining the importance from d1 and d2-----
importances_d1d2 <- full_join(

  gc@test_consensus@mlp$importances %>%
    mutate(d1_signal = ifelse(Importance > 0, "Positive", "Negative")) %>%
    select(Variable, Importance_D1 = Importance, d1_signal)
  ,

  tc@mlp$importances %>%
    mutate(d2_signal = ifelse(Importance > 0, "Positive", "Negative")) %>%
```

```

    select(Variable, Importance_d2 = Importance, d2_signal)
  ) %>%
  mutate(same_signal = d1_signal == d2_signal,
         sum_importance = abs(Importance_D1 + Importance_d2)) %>%
  arrange(desc(same_signal), desc(sum_importance))

# Pull genes that have the same importance direction
genes_for_valcons <- importances_d1d2 %>%
  filter(same_signal) %>%
  pull(Variable)

```

5.2 Use of valConsensus() to identify minimal gene signature

```

# valConsensus
vc <- valConsensus(
  df.count = pp_d3@processed$adjusted.data,
  gene.list = genes_for_valcons,
  class = pp_d3@metadata$class
)

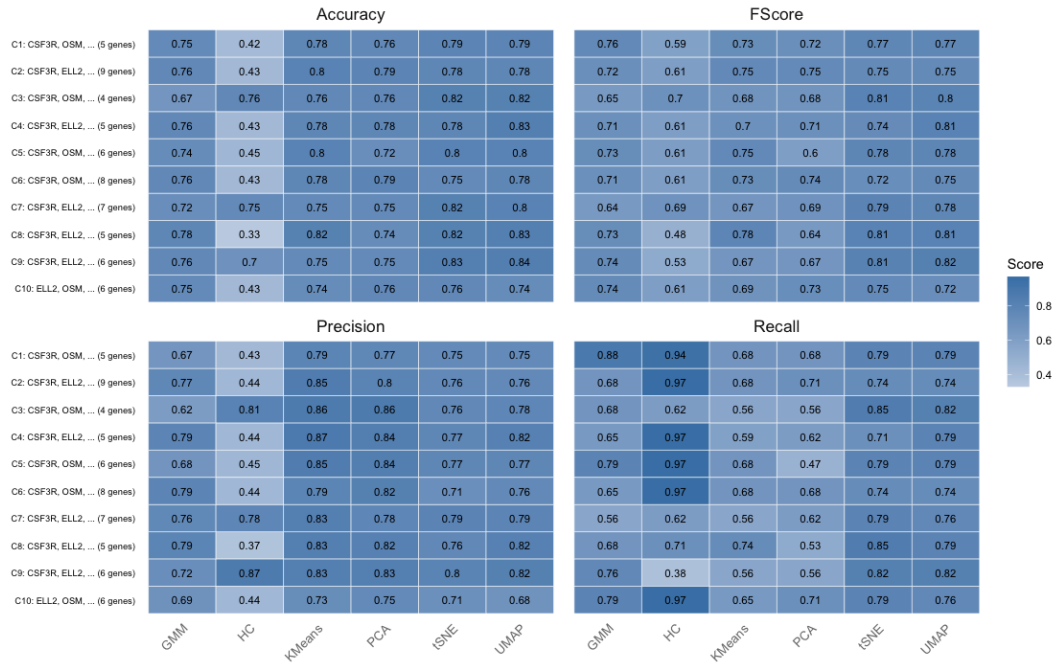
```

The output is a list containing clustering results, evaluation metrics, and ranked gene combinations, which can be visualized using the `plotTopMetrics()` function that creates a heatmap to visualize the performance of different clustering and dimensionality reduction models across multiple evaluation metrics, for each gene combination.

Each combination is labeled with a concise and readable gene summary on the y-axis, prefixed by a combination number (C1, C2, ...):

```
plotTopMetrics(vc$topCombinations)
```

Top Model Metrics



6 Independent Unsupervised Validation on Dataset 4

To assess the discriminative capacity of the minimal core signature in a fully unsupervised setting, the expression matrix of Dataset 4 was restricted to the five signature genes (CSF3R, OSM, BACE2, HK2, IL15RA) and projected onto a two-dimensional t-SNE embedding.

Among the six clustering methods evaluated in the Minimal Signature Identification Phase, k-means on t-SNE embeddings had yielded the most consistent performance across gene combinations and metrics. It was therefore applied as the sole method on Dataset 4, with $k = 2$ and class labels used exclusively a posteriori to evaluate clustering performance.

```
genes <- c("CSF3R", "OSM", "BACE2", "HK2", "IL15RA")

X <- pp_d4@processed$adjusted.data %>% as.data.frame() %>%
  mutate(across(everything(), as.numeric)) %>% as.matrix() %>% .[, genes] %>% scale()

tsne_out <- Rtsne(X, perplexity = 10, dims = 2, verbose = TRUE, check_duplicates = FALSE)

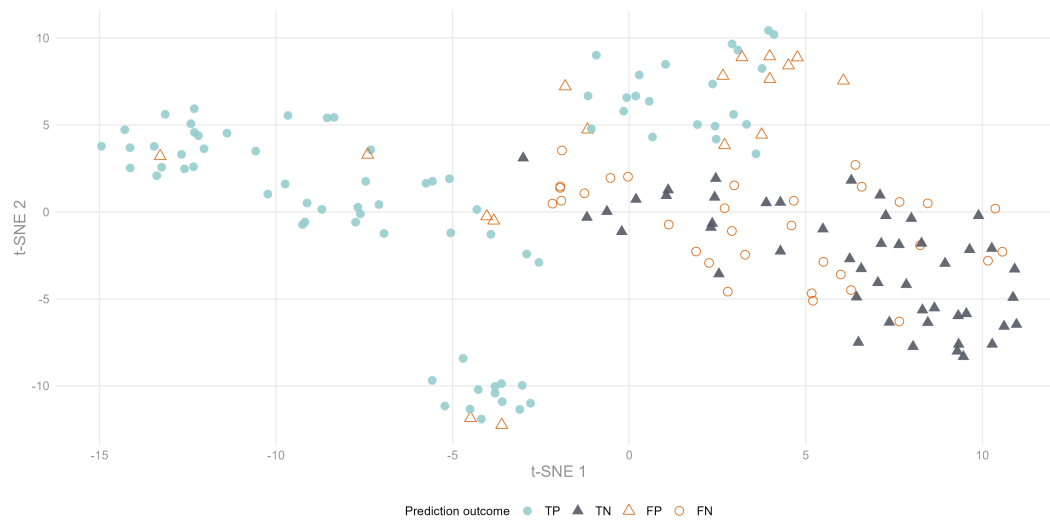
tsne_df <- as.data.frame(tsne_out$Y) %>%
  setNames(c("tSNE1", "tSNE2")) %>%
  mutate(group = as.character(pp_d4@metadata$class),
         predicted = as.character(kmeans(tsne_out$Y, centers = 2, iter.max = 100, algorithm = "MacQueen"),
         outcome = case_when(group == "1" & predicted == "1" ~ "TP",
                             group == "0" & predicted == "0" ~ "TN",
                             group == "0" & predicted == "1" ~ "FP",
                             group == "1" & predicted == "0" ~ "FN"),
         outcome = factor(outcome, levels = c("TP", "TN", "FP", "FN")))

color_values <- c(TP = "#96CDcf", TN = "#535965", FP = "#D55E00", FN = "#D55E00")

shape_values <- c(TP = 16, TN = 17, FP = 2, FN = 1)

tsne_plot <- ggplot(tsne_df, aes(tSNE1, tSNE2)) +
  geom_point(aes(color = outcome, shape = outcome), size = 3, alpha = 0.9) +
  scale_color_manual(values = color_values, name = "Prediction outcome") +
  scale_shape_manual(values = shape_values, name = "Prediction outcome") +
  labs(x = "t-SNE 1", y = "t-SNE 2") +
  theme_minimal() +
  theme(legend.box.just = "right",
        legend.position = "bottom",
        legend.title = element_text(size = 10),
        legend.text = element_text(size = 10),
        axis.text = element_text(size = 10, color = "#8f8f8f"),
        axis.title = element_text(size = 13, color = "#8f8f8f"),
        panel.grid.minor = element_blank())

tsne_plot
```



```
# Compute metrics from outcome counts
outcome_counts <- table(tsne_df$outcome)

TP <- as.numeric(outcome_counts["TP"])
TN <- as.numeric(outcome_counts["TN"])
FP <- as.numeric(outcome_counts["FP"])
FN <- as.numeric(outcome_counts["FN"])

accuracy <- (TP + TN) / (TP + TN + FP + FN)
precision <- TP / (TP + FP)
recall <- TP / (TP + FN)
f_meas <- 2 * (precision * recall) / (precision + recall)

metrics_df <- data.frame(
  Metric = c("Accuracy", "Precision", "Recall", "F1-score"),
  Value = round(c(accuracy, precision, recall, f_meas), 3)
)

knitr::kable(metrics_df, align = "lr")

rm(outcome_counts, TP, TN, FP, FN, accuracy, precision, recall, f_meas)
```

Metric	Value
Accuracy	0.72
Precision	0.82
Recall	0.71
F1-score	0.76

7 Session Info

```
devtools::session_info()
#> Warning in system2("quarto", "-V", stdout = TRUE, env = paste0("TMPDIR=", :
#> running command '"quarto"
#> TMPDIR=C:/Users/gabri/AppData/Local/Temp/RtmpKareks/file256833334b6a -V' had
#> status 1
#> - Session info -----
#> setting value
#> version R version 4.4.3 (2025-02-28 ucrt)
#> os Windows 11 x64 (build 26200)
#> system x86_64, mingw32
#> ui RTerm
#> language en
#> collate Portuguese_Brazil.utf8
#> ctype Portuguese_Brazil.utf8
#> tz Europe/Berlin
#> date 2026-04-17
#> pandoc 3.6.3 @ C:/Program Files/RStudio/resources/app/bin/quarto/bin/tools/ (via rmarkdown)
#> quarto NA @ C:/PROGRA~1/RStudio/RESOUR~1/app/bin/quarto/bin/quarto.exe
#>
#> - Packages -----
#> package * version date (UTC) lib source
#> abind 1.4-8 2024-09-12 [1] CRAN (R 4.4.1)
#> annotate 1.84.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> AnnotationDbi 1.68.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> assertthat 0.2.1 2019-03-21 [1] CRAN (R 4.4.3)
#> baguette 1.1.0 2025-01-28 [1] CRAN (R 4.4.3)
#> Biobase * 2.66.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> BiocGenerics * 0.52.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> BiocParallel 1.40.2 2025-06-04 [1] Bioconductor
#> Biostrings 2.74.1 2024-12-16 [1] Bioconductor 3.20 (R 4.4.2)
#> bit 4.6.0 2025-03-06 [1] CRAN (R 4.4.3)
#> bit64 4.6.0-1 2025-01-16 [1] CRAN (R 4.4.2)
#> blob 1.2.4 2023-03-17 [1] CRAN (R 4.4.1)
#> bonsai 0.4.0 2025-06-25 [1] CRAN (R 4.4.3)
#> Boruta 9.0.0 2025-07-22 [1] CRAN (R 4.4.3)
#> bslib 0.9.0 2025-01-30 [1] CRAN (R 4.4.3)
#> C50 0.2.0 2025-04-03 [1] CRAN (R 4.4.3)
#> ca 0.71.1 2020-01-24 [1] CRAN (R 4.4.3)
#> cachem 1.1.0 2024-05-16 [1] CRAN (R 4.4.1)
#> class 7.3-23 2025-01-01 [2] CRAN (R 4.4.3)
#> cli 3.6.5 2025-04-23 [1] CRAN (R 4.4.3)
#> codetools 0.2-20 2024-03-31 [2] CRAN (R 4.4.3)
#> colino 0.0.1 2025-04-28 [1] Github (stevenpawley/colino@9a786ce)
#> colorspace 2.1-1 2024-07-26 [1] CRAN (R 4.4.1)
#> corpcor 1.6.10 2021-09-16 [1] CRAN (R 4.4.0)
#> couplot * 1.2.0 2025-07-07 [1] CRAN (R 4.4.3)
#> crayon 1.5.3 2024-06-20 [1] CRAN (R 4.4.1)
#> Cubist 0.5.0 2025-04-03 [1] CRAN (R 4.4.3)
#> data.table 1.17.8 2025-07-10 [1] CRAN (R 4.4.3)
#> DBI 1.3.0 2026-02-25 [1] CRAN (R 4.4.3)
#> DelayedArray 0.32.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
```

```

#> dendextend          1.19.1    2025-07-15 [1] CRAN (R 4.4.3)
#> DESeq2               * 1.46.0    2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> devtools            2.4.5      2022-10-11 [1] CRAN (R 4.4.1)
#> dials                1.4.2      2025-09-04 [1] CRAN (R 4.4.3)
#> DiceDesign           1.10       2023-12-07 [1] CRAN (R 4.4.3)
#> dichromat            2.0-0.1    2022-05-02 [1] CRAN (R 4.4.0)
#> digest               0.6.37     2024-08-19 [1] CRAN (R 4.4.1)
#> dplyr                * 1.2.0     2026-02-03 [1] CRAN (R 4.4.3)
#> edgeR                4.4.2      2025-01-27 [1] Bioconductor 3.20 (R 4.4.2)
#> ellipse              0.5.0      2023-07-20 [1] CRAN (R 4.4.2)
#> ellipsis             0.3.2      2021-04-29 [1] CRAN (R 4.4.1)
#> evaluate             1.0.5      2025-08-27 [1] CRAN (R 4.4.3)
#> farver               2.1.2      2024-05-13 [1] CRAN (R 4.4.1)
#> fastmap              1.2.0      2024-05-15 [1] CRAN (R 4.4.1)
#> forcats              * 1.0.0     2023-01-29 [1] CRAN (R 4.4.1)
#> foreach              1.5.2      2022-02-02 [1] CRAN (R 4.4.1)
#> Formula              1.2-5      2023-02-24 [1] CRAN (R 4.4.0)
#> fs                   1.6.6      2025-04-12 [1] CRAN (R 4.4.3)
#> FSelectorRcpp        0.3.13     2024-10-02 [1] CRAN (R 4.4.3)
#> furr                 0.3.1      2022-08-15 [1] CRAN (R 4.4.2)
#> future               1.67.0     2025-07-29 [1] CRAN (R 4.4.3)
#> future.apply         1.20.0     2025-06-06 [1] CRAN (R 4.4.3)
#> genefilter           1.88.0     2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> generics             0.1.4      2025-05-09 [1] CRAN (R 4.4.3)
#> GenomeInfoDb         * 1.42.3     2025-01-27 [1] Bioconductor 3.20 (R 4.4.2)
#> GenomeInfoDbData     1.2.13     2024-12-03 [1] Bioconductor
#> GenomicRanges        * 1.58.0     2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> ggplot2              * 3.5.2      2025-04-09 [1] CRAN (R 4.4.3)
#> ggrepel              * 0.9.6      2024-09-07 [1] CRAN (R 4.4.1)
#> globals              0.18.0     2025-05-08 [1] CRAN (R 4.4.3)
#> glue                 1.8.0      2024-09-30 [1] CRAN (R 4.4.1)
#> gower                1.0.2      2024-12-17 [1] CRAN (R 4.4.2)
#> GPfit                1.0-9      2025-04-12 [1] CRAN (R 4.4.3)
#> gridExtra            2.3        2017-09-09 [1] CRAN (R 4.4.1)
#> gtable               0.3.6      2024-10-25 [1] CRAN (R 4.4.2)
#> hardhat              1.4.2      2025-08-20 [1] CRAN (R 4.4.3)
#> heatmaply            1.6.0      2025-07-12 [1] CRAN (R 4.4.3)
#> hms                  1.1.3      2023-03-21 [1] CRAN (R 4.4.1)
#> htmltools            0.5.8.1    2024-04-04 [1] CRAN (R 4.4.1)
#> htmlwidgets          1.6.4      2023-12-06 [1] CRAN (R 4.4.1)
#> httpuv               1.6.16     2025-04-16 [1] CRAN (R 4.4.3)
#> httr                 1.4.7      2023-08-15 [1] CRAN (R 4.4.1)
#> igraph               2.1.4      2025-01-23 [1] CRAN (R 4.4.2)
#> inum                 1.0-5      2023-03-09 [1] CRAN (R 4.4.3)
#> ipred                0.9-15     2024-07-18 [1] CRAN (R 4.4.2)
#> IRanges              * 2.40.1     2024-12-05 [1] Bioconductor 3.20 (R 4.4.2)
#> iterators            1.0.14     2022-02-05 [1] CRAN (R 4.4.1)
#> jquerylib            0.1.4      2021-04-26 [1] CRAN (R 4.4.1)
#> jsonlite             2.0.0      2025-03-27 [1] CRAN (R 4.4.3)
#> KEGGREST             1.46.0     2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> knitr                1.50       2025-03-16 [1] CRAN (R 4.4.3)
#> labeling             0.4.3      2023-08-29 [1] CRAN (R 4.4.0)
#> later                1.4.4      2025-08-27 [1] CRAN (R 4.4.3)

```

```

#> lattice                0.22-7      2025-04-02 [2] CRAN (R 4.4.3)
#> lava                   1.8.1       2025-01-12 [1] CRAN (R 4.4.2)
#> lazyeval               0.2.2       2019-03-15 [1] CRAN (R 4.4.1)
#> lhs                    1.2.0       2024-06-30 [1] CRAN (R 4.4.3)
#> libcoin                1.0-10      2023-09-27 [1] CRAN (R 4.4.3)
#> lifecycle              1.0.5       2026-01-08 [1] CRAN (R 4.4.3)
#> limma                  3.62.2      2025-01-09 [1] Bioconductor 3.20 (R 4.4.2)
#> listenv                 0.9.1       2024-01-29 [1] CRAN (R 4.4.2)
#> locfit                 1.5-9.12    2025-03-05 [1] CRAN (R 4.4.3)
#> lubridate              * 1.9.4     2024-12-08 [1] CRAN (R 4.4.2)
#> magrittr               2.0.3       2022-03-30 [1] CRAN (R 4.4.1)
#> MASS                   7.3-65      2025-02-28 [2] CRAN (R 4.4.3)
#> Matrix                 1.7-3       2025-03-11 [2] CRAN (R 4.4.3)
#> MatrixGenerics        * 1.18.1    2025-01-09 [1] Bioconductor 3.20 (R 4.4.2)
#> matrixStats           * 1.5.0     2025-01-07 [1] CRAN (R 4.4.2)
#> mclust                 6.1.1       2024-04-29 [1] CRAN (R 4.4.3)
#> memoise                2.0.1       2021-11-26 [1] CRAN (R 4.4.1)
#> mgcv                   1.9-3       2025-04-04 [1] CRAN (R 4.4.3)
#> mime                   0.13        2025-03-17 [1] CRAN (R 4.4.3)
#> miniUI                 0.1.2       2025-04-17 [1] CRAN (R 4.4.3)
#> mixOmics               6.30.0     2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> mvtnorm                1.3-3       2025-01-10 [1] CRAN (R 4.4.2)
#> NeuralNetTools         1.5.3       2022-01-06 [1] CRAN (R 4.4.3)
#> nlme                   3.1-168     2025-03-31 [2] CRAN (R 4.4.3)
#> nnet                   7.3-20      2025-01-01 [1] CRAN (R 4.4.2)
#> otel                   0.2.0       2025-08-29 [1] CRAN (R 4.4.3)
#> parallely              1.45.1     2025-07-24 [1] CRAN (R 4.4.3)
#> parsnip                1.3.2       2025-05-28 [1] CRAN (R 4.4.3)
#> partykit               1.2-24      2025-05-02 [1] CRAN (R 4.4.3)
#> pillar                 1.11.0     2025-07-04 [1] CRAN (R 4.4.3)
#> pkgbuild               1.4.8       2025-05-26 [1] CRAN (R 4.4.3)
#> pkgconfig              2.0.3       2019-09-22 [1] CRAN (R 4.4.1)
#> pkgload                1.4.0       2024-06-28 [1] CRAN (R 4.4.1)
#> plotly                 4.11.0     2025-06-19 [1] CRAN (R 4.4.3)
#> plsmod                 1.0.0       2022-09-06 [1] CRAN (R 4.4.3)
#> plyr                   1.8.9       2023-10-02 [1] CRAN (R 4.4.1)
#> png                    0.1-8       2022-11-29 [1] CRAN (R 4.4.0)
#> praznik                11.0.0     2022-05-20 [1] CRAN (R 4.4.3)
#> prodlim                2025.04.28 2025-04-28 [1] CRAN (R 4.4.3)
#> profvis                0.4.0       2024-09-20 [1] CRAN (R 4.4.1)
#> promises               1.3.3       2025-05-29 [1] CRAN (R 4.4.3)
#> purrr                  * 1.1.0     2025-07-10 [1] CRAN (R 4.4.3)
#> R6                     2.6.1       2025-02-15 [1] CRAN (R 4.4.3)
#> rARPACK                0.11-0     2016-03-10 [1] CRAN (R 4.4.3)
#> RColorBrewer           1.1-3       2022-04-03 [1] CRAN (R 4.4.0)
#> Rcpp                   1.1.0       2025-07-02 [1] CRAN (R 4.4.3)
#> readr                  * 2.1.5     2024-01-10 [1] CRAN (R 4.4.1)
#> recipes                1.3.1       2025-05-21 [1] CRAN (R 4.4.3)
#> registry               0.5-1       2019-03-05 [1] CRAN (R 4.4.0)
#> remotes                2.5.0       2024-03-17 [1] CRAN (R 4.4.1)
#> reshape2              1.4.4       2020-04-09 [1] CRAN (R 4.4.1)
#> rlang                  1.1.7       2026-01-09 [1] CRAN (R 4.4.3)
#> rmarkdown              2.29        2024-11-04 [1] CRAN (R 4.4.2)

```

```

#> rpart 4.1.24 2025-01-07 [2] CRAN (R 4.4.3)
#> rsample 1.3.0 2025-04-02 [1] CRAN (R 4.4.3)
#> RSpectra 0.16-2 2024-07-18 [1] CRAN (R 4.4.2)
#> RSQLite 2.4.3 2025-08-20 [1] CRAN (R 4.4.3)
#> rstudioapi 0.17.1 2024-10-22 [1] CRAN (R 4.4.2)
#> Rtsne * 0.17 2023-12-07 [1] CRAN (R 4.4.3)
#> rules 1.0.2 2023-03-08 [1] CRAN (R 4.4.3)
#> S4Arrays 1.6.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> S4Vectors * 0.44.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> sass 0.4.10 2025-04-11 [1] CRAN (R 4.4.3)
#> scales * 1.4.0 2025-04-24 [1] CRAN (R 4.4.3)
#> seriation 1.5.8 2025-08-20 [1] CRAN (R 4.4.3)
#> sessioninfo 1.2.3 2025-02-05 [1] CRAN (R 4.4.3)
#> shiny 1.11.1 2025-07-03 [1] CRAN (R 4.4.3)
#> SparseArray 1.6.2 2025-02-20 [1] Bioconductor 3.20 (R 4.4.2)
#> statmod 1.5.0 2023-01-06 [1] CRAN (R 4.4.2)
#> stringi 1.8.7 2025-03-27 [1] CRAN (R 4.4.3)
#> stringr * 1.5.1 2023-11-14 [1] CRAN (R 4.4.1)
#> SummarizedExperiment * 1.36.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> survival 3.8-3 2024-12-17 [1] CRAN (R 4.4.2)
#> sva 3.54.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> TENTACLES * 0.1.0 2025-10-30 [1] Github (Giomu/TENTACLES@199e3f7)
#> tibble * 3.3.0 2025-06-08 [1] CRAN (R 4.4.3)
#> tidyr * 1.3.1 2024-01-24 [1] CRAN (R 4.4.1)
#> tidyselect 1.2.1 2024-03-11 [1] CRAN (R 4.4.1)
#> tidyverse * 2.0.0 2023-02-22 [1] CRAN (R 4.4.1)
#> tiff * 0.1-12 2023-11-28 [1] CRAN (R 4.4.0)
#> timechange 0.3.0 2024-01-18 [1] CRAN (R 4.4.1)
#> timeDate 4041.110 2024-09-22 [1] CRAN (R 4.4.1)
#> TSP 1.2-5 2025-05-27 [1] CRAN (R 4.4.3)
#> tune 1.3.0 2025-02-21 [1] CRAN (R 4.4.3)
#> tzdb 0.5.0 2025-03-15 [1] CRAN (R 4.4.3)
#> UCSC.utils 1.2.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> urlchecker 1.0.1 2021-11-30 [1] CRAN (R 4.4.1)
#> usethis 3.2.0 2025-08-28 [1] CRAN (R 4.4.3)
#> vctrs 0.7.1 2026-01-23 [1] CRAN (R 4.4.3)
#> viridis 0.6.5 2024-01-29 [1] CRAN (R 4.4.1)
#> viridisLite 0.4.2 2023-05-02 [1] CRAN (R 4.4.1)
#> webshot 0.5.5 2023-06-26 [1] CRAN (R 4.4.3)
#> withr 3.0.2 2024-10-28 [1] CRAN (R 4.4.2)
#> workflows 1.3.0 2025-08-27 [1] CRAN (R 4.4.3)
#> xfun 0.53 2025-08-19 [1] CRAN (R 4.4.3)
#> XML 3.99-0.19 2025-08-22 [1] CRAN (R 4.4.3)
#> xtable 1.8-4 2019-04-21 [1] CRAN (R 4.4.1)
#> XVector 0.46.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#> yaml 2.3.10 2024-07-26 [1] CRAN (R 4.4.1)
#> yardstick 1.3.2 2025-01-22 [1] CRAN (R 4.4.3)
#> zlibbioc 1.52.0 2024-10-29 [1] Bioconductor 3.20 (R 4.4.1)
#>
#> [1] C:/Users/gabri/AppData/Local/R/win-library/4.4
#> [2] C:/Program Files/R/R-4.4.3/library
#> * -- Packages attached to the search path.
#>

```

#> -----